AgriDax Data Platform
Target GCP Architecture

Architecture proposal derived from the current validated implementation

Prepared by
Andres Olguin
Data Engineer

October 2026 • Version 1.0

The target design preserves AgriDax's delivery evidence, revision semantics, temporal history, quality controls, and coordinated publication while adapting execution and access control to Google Cloud.

TARGET — NOT DEPLOYED

Status: proposed architecture. The repository implements a local PostgreSQL platform. No GCP resource, cloud operating history, benchmark, or production SLA is claimed.

Source baseline

https://github.com/andresolguin/agridax-data-platform

Inspected revision: 42b0a7fba760119cc3f014191859f507208043fa

All 125 repository files were retrieved recursively at this revision and checked against their Git blob hashes. The supporting final audit and native acceptance evidence inform the validated baseline; they do not replace the code review.

Primary recommendation

Use immutable accepted ingestion batches, a persisted run input set, isolated dbt outputs, and protected release generations in BigQuery. Activate all five consumer marts through one control row, and require an explicit release pin for reports that issue multiple queries.

01 Executive Summary

AgriDax is a batch data platform for farm, field, agricultural event, weather, and sustainability data. Its strongest architectural feature is the separation of delivered evidence, canonical decisions, historical state, and consumer visibility. The target GCP design retains those boundaries rather than reproducing PostgreSQL's session locks and table replacement mechanics.

The recommended execution path is Cloud Storage landing → Cloud Run ingestion jobs → BigQuery RAW and operational metadata → Cloud Workflows → a Cloud Run job running dbt Core with dbt-bigquery → private candidate marts → stored-cohort validation → versioned publication → authorized views and BI consumers. Artifact Registry supplies pinned images; dedicated identities, Cloud Logging, and Cloud Monitoring support controlled execution. Secret Manager is needed only for external credentials that an implemented connector actually requires.

A release is a complete, validated generation of dim_farms, dim_fields, fct_crop_cycles, fct_field_daily_performance, and fct_field_events. Its five physical copies are kept in a private release_store dataset. A short BigQuery transaction changes ops.current_release and records the activation event. Copying or validating a failed generation does not change consumer visibility. A report spanning several queries must resolve and retain one release_id; reading current views independently is only a per-query consistency contract.

Ingestion can continue during transformation because each run persists the exact accepted batch IDs it will process. Changed keys come from batches not yet represented by confirmed processing state. Winner resolution still considers every eligible revision of those keys. A timestamp cutoff alone is insufficient: a batch accepted after input capture must be discovered by membership difference even when its timestamp is older than the next cutoff.

The initial cloud implementation keeps one fenced transformation coordinator and permits concurrent ingestion preparation. BigQuery transactions protect narrow control operations; isolated attempt outputs keep an abandoned worker from changing confirmed state. This is an application protocol that must be implemented and tested, not a claim of native BigQuery advisory locking.

Evidence and claim boundaries

Current validated implementation	Target GCP architecture
Python, PostgreSQL, Docker/Compose, PowerShell entry points, and dbt-postgres are present in the inspected repository.	Cloud Storage, BigQuery, Cloud Run Jobs, Workflows, IAM, and Terraform are proposed. No cloud deployment files or deployed resources were found.

Current validated implementation	Target GCP architecture
The supplied final audit declares READY FOR FIRST COMMIT AND PUBLIC GITHUB and closes B01. Supplied native acceptance evidence reports 142 PASS, 0 FAIL, and 0 NOT RUN.	Cloud acceptance must independently test adapter semantics, batch acceptance, fenced progress, snapshot commits, release activation, IAM denial paths, and reader pins.
The repository demonstrates functional behavior on fixtures and explicit acceptance scenarios.	Volume, service cadence, costs, throughput, production concurrency, SLA, RPO, and RTO remain unmeasured.

Reading guide: Sections 2–6 establish the baseline and data flow; 7–11 define BigQuery state and publication; 12–18 cover operations and risks; 19–24 cover migration decisions and delivery phases; 25 provides an architecture design walkthrough; 26–27 provide official references and repository traceability.

02 Current AgriDax Architecture and Invariants

CURRENT — VALIDATED

Current validated implementation. PostgreSQL holds append-only RAW deliveries and operational run/checkpoint metadata. Python manages preflight checks, role administration, ingestion, the dbt invocation, provenance checks, validation, and promotion. dbt builds ten staging models, eight intermediate models, five marts, and an observed field snapshot. The five marts are candidate relations in analytics until publication succeeds. [R1–R8]

A managed writer takes PostgreSQL advisory locks, holds an ingestion horizon stable, runs the full required DAG, checks artifacts and canonical completeness, and publishes through a REPEATABLE READ transaction. Transactional DELETE + INSERT refreshes all five stable published tables and their release marker together. Table identity and compatible dependent readers are preserved. A failed build or promotion leaves the preceding published cohort available. This behavior is implemented locally; BigQuery requires different control and publication contracts.

Invariant	Current evidence	Target obligation
Delivery evidence	RAW has physical delivery IDs and permits retries and revisions rather than business-key deduplication.	Retain original objects, batch manifests, and every distinct physical delivery; retrying the same accepted batch must not duplicate it.
Canonical correctness	Event and weather winners rank producer revision, ingestion timestamp, then delivery ID. Invalid latest winners remain canonical.	Resolve all versions for changed keys and retain invalid winners instead of falling back to an older trusted value.

Invariant	Current evidence	Target obligation
Temporal separation	Effective field-state history uses source effective dates; snap_fields records observed current-state changes.	Maintain separate business and observation clocks, and preserve observed history during recovery.
Progress independence	Checkpoints describe inspected delivery horizons independently of winning rows.	Record processed batch membership and confirmed canonical state even when batches are invalid or do not change a winner.
Consumer isolation	Candidate analytics relations are separate from published tables; all five marts pass publication gates.	Keep working and incomplete release data private, then activate one validated five-mart generation.
Recovery and access	Bounded recovery, provenance validation, and separate ingest/dbt/publish/reader/admin identities are implemented.	Preserve boundaries with durable run plans, controlled routines, dedicated service accounts, and cloud acceptance tests.

Domain semantics that must remain visible

Events use the normalized pair (source_system, source_event_id). Weather uses (field_id, observed_at), including competing provider deliveries. Field states use (field_id, effective_from); interval history and current-as-of state are derived after revision selection. These are AgriDax contracts, not interchangeable generic warehouse grains.

Crop cycles start at eligible planting events. The first eligible harvest before the next planting closes a cycle; a later planting can supersede an unharvested cycle. Simultaneous distinct plantings are rejected by a quality gate. Yield uses the first harvest quantity divided by whole-field area at planting. It does not represent planted-area yield or all harvested production. Weather rainfall is treated as interval rainfall; irrigation depth is additive, not area weighted.

Daily facts use UTC business dates and preserve NULL for absent measurements versus zero for reported zero. Presence flags do not assert full-day coverage. Sustainability records represent reported status; the platform does not infer a complete validity timeline. These limitations remain part of the target consumer contract. [R2, R4, R8]

Validated baseline and remaining limitations

The supporting audit closes the native environment blocker with a complete Windows 11, Python 3.12.10, PowerShell 7, Docker, PostgreSQL 16.15, and dbt-postgres acceptance run. It also notes non-blocking gaps: historical field name/status validation is less complete than current-state validation; the local publisher can read more analytics relations than it needs; contradictory artifact environment-key casing is not explicitly rejected; and some historical readiness wording lags the final evidence. The cloud roadmap addresses these as explicit migration work. [E1–E2]

- Keep native accepted RAW and immutable object evidence; mutable external files would undermine stable run inputs.
- Keep incomplete generations private behind published authorized interfaces.
- Persist batch membership; ingestion timestamps alone are not a complete checkpoint.
- Protect committed canonical state and snapshots from direct dbt writes.
- Use controlled acceptance/publication routines; table write permission is broader than SQL INSERT.
- Add release pins for reports with multiple queries; current views provide per-query consistency.
- Keep one fenced transformation coordinator while ingestion continues independently.

No requirement in the current platform demonstrates a need for Pub/Sub, Dataflow, Composer, Dataproc, or GKE. The design omits them. Future push ingestion, sustained streaming, or operational constraints would require a new measured decision rather than adding services for presentation value. [R5]

04 Architecture Diagram

Target GCP architecture. Solid arrows show data or materialization; dashed arrows show execution and control. The diagram groups committed state inside the transformation boundary and places operational decisions outside the consumer interface.

Target GCP architecture

PROPOSED • Immutable batches, isolated processing, validated releases

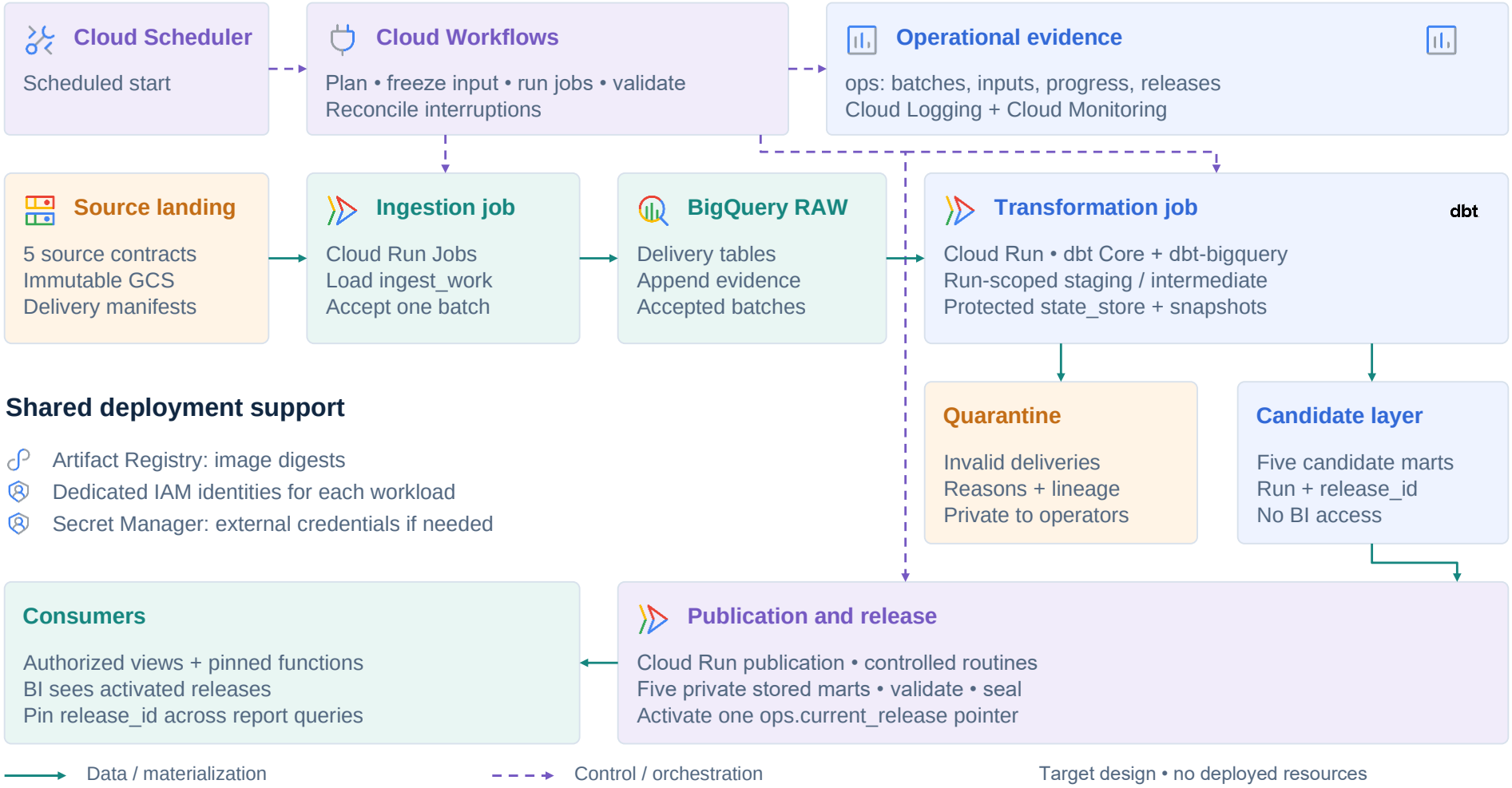


Figure 1. Proposed architecture. RAW acceptance, confirmed processing state, observed snapshots, and active publication use separate durable records. All consumer reads pass through the published interface.

05 Component-by-Component Responsibilities

Component	AgriDax responsibility	Boundary
Cloud Storage landing	Store original five-stream deliveries, immutable manifests, and retained run artifacts.	Evidence and replay; it is not a mutable transformation source.
Cloud Run ingestion	Validate structural contracts, preserve source values, prepare load tables, and request batch acceptance.	Cannot modify or delete accepted RAW directly.
BigQuery RAW and ingest_work	Separate temporary load preparation from accepted delivery evidence.	Only accepted batches enter run input sets.
Cloud Workflows and Scheduler	Create or resume run plans, invoke jobs, poll results, and reconcile interruptions.	Control plane; no data arrays or unrestricted warehouse mutation.
Cloud Run dbt job	Resolve changed keys, build effective histories, stage observed changes, and build five candidate marts.	Read frozen inputs and committed state; write attempt-specific outputs.
Publication job and controlled routines	Copy, validate, seal, and activate complete release generations.	No direct edits to active or sealed releases.
Published views and functions	Serve the five current marts and retained activated releases selected by a pin.	BI reads neither RAW nor candidate or private release tables.
Artifact Registry	Distribute ingestion, dbt, and publication images by digest.	A digest identifies code; a mutable latest tag is not run provenance.
IAM and Secret Manager	Separate workload identities; supply external credentials only if a connector needs them.	Cloud access uses attached identities and ADC, not stored GCP keys.
Cloud Logging and Monitoring	Correlate failures and pipeline evidence; report staleness and operational anomalies.	Logs support investigation; durable warehouse evidence remains authoritative.

The additional ingest_work, state_store, and release_store datasets are justified by distinct mutation rights. They do not introduce extra GCP services. Run outputs and committed decisions must not share a broad writable boundary.

06 End-to-End Data Flow

1. Receive a source delivery. Write original bytes under a unique landing object name with a creation precondition. Record source, schema version, object generation, content checksum, byte size, row count, and receipt time.
2. Prepare a batch. The ingestion job derives a stable batch_id and delivery IDs, parses explicitly typed columns without discarding source literals, and loads a deterministic temporary ingest_work destination.
3. Accept the batch. A controlled BigQuery procedure verifies its manifest and prepared rows, then commits RAW rows and accepted-batch metadata in one transaction. An existing matching acceptance returns the original result.
4. Plan the run. Workflows requests a durable run_id, mode, bounds, code/image hashes, initial release_id, and expected predecessor. A fenced coordinator captures H_r and the exact accepted batch set B_r in ops.run_input_batches.
5. Resolve changed keys. Compare B_r with confirmed processed-batch membership for each stream. Use those deliveries to find affected keys, then inspect every eligible version of those keys inside B_r.
6. Confirm internal progress. Validate canonical output and snapshot changes. Controlled procedures seal a canonical state generation, record inspected batch membership, and commit observed snapshot changes idempotently. Consumer visibility is still unchanged.
7. Build and validate candidates. dbt produces all five marts under the run and attempt namespace. Complete DAG, reconciliation, schema, grain, relationship, freshness, temporal, and metric gates must pass.
8. Prepare the release. Copy each validated candidate into private release_store with release_id. Validate the stored cohort and its evidence, then seal it against further writes.
9. Activate. A short transaction compares the predecessor and fence, updates the single current-release row, and inserts an activation event. Consumers now resolve the new complete generation.
10. Retain and reconcile. Persist artifacts and lineage; release the coordinator. Cleanup later removes only unprotected retired generations. Interrupted executions resume from durable decisions rather than assuming they failed.

Identifier and evidence contract

Identifier or state	Meaning and relationship
batch_id / delivery_id	batch_id identifies a source object generation and contract. delivery_id identifies its row ordinal. Same-batch retries retain both; a new physical source delivery receives a new batch identity.
run_id / attempt_id	run_id identifies a durable processing plan. attempt_id identifies disposable physical outputs from one execution attempt. A retry cannot overwrite another attempt’s committed state.

Identifier or state	Meaning and relationship
release_id	Identifies the proposed five-mart consumer generation. It references the successful run/attempt and evidence. Failed runs need not create a visible release.
B_r / H_r	B_r is the persisted accepted input membership; H_r is a UTC processing/ business cutoff captured with that input. Every source and validation uses the same membership.
state_generation_id / progress	Identifies validated canonical caches plus their processed batch membership. Progress must reference recoverable state and never be inferred from the latest winning row.
observation_id / observed head	Identifies an idempotent warehouse observation committed for a run. It advances observed history independently of consumer activation.
activation_epoch / request_id	A monotonic activation epoch supports compare-and-set. request_id identifies one activation operation so ambiguous responses can be reconciled without replaying an obsolete promotion.
provenance	Git commit, image digest, dependency lock, dbt invocation IDs, input-set hash, schema/contract versions, job IDs and locations, validation reports, and five stored-mart manifests.

BigQuery holds large batch membership sets and manifests. Workflow arguments carry IDs and references, not an unbounded list of source rows or batches. A sorted manifest digest, or a documented hierarchical digest when needed, makes input membership auditable without pretending that a hash itself is a data snapshot.

07 BigQuery Dataset and Layer Design

TARGET — NOT DEPLOYED

Target GCP architecture. The following is a concrete logical layout within a configured project and location. Dataset names are proposed, not existing resources. BigQuery does not nest schemas; analytics_staging and analytics_intermediate replace the conceptual analytics/staging and analytics/intermediate paths.

Dataset	Proposed contents	Write authority and lifecycle
raw	farms, fields, field_events, weather_ observations, sustainability_practices; source literals, typed fields, delivery_id, batch_id, ingested_at, revision timestamps.	Accepted append evidence through static procedures. No default expiry until an evidence-retention policy is approved.

Dataset	Proposed contents	Write authority and lifecycle
ingest_work	Per-batch load preparation, parsing diagnostics, and explicit load destinations.	Ingestion workload writes; temporary outputs expire only after acceptance or retained failure evidence.
analytics_ staging	Attempt-specific normalized delivery/state relations and trusted projections.	dbt writes disposable run outputs. All source scans join frozen input membership.
analytics_ intermediate	Attempt-specific winners, field intervals, crop pairing, and metric preparation.	dbt writes working outputs; it cannot edit protected confirmed caches.
state_store	Versioned confirmed canonical winner caches, including invalid winners, and field state caches.	Controlled commit routines write. dbt reads the selected base generation; checkpoints reference this state.
quarantine	Physical invalid delivery rows, rejection reasons, batch lineage, and relevant run evidence.	Controlled append from validated rejection outputs; delivery identity prevents repeated quarantine copies.
snapshots	Confirmed snap_fields observation history and observation commit records.	Controlled SCD commit; dbt stages changes rather than mutating shared committed history directly.
candidate	Five marts per run/attempt, plus staged snapshot/validation outputs.	dbt writes; no BI access. Unsealed attempts are disposable after reconciliation.
release_store	Five physical mart tables with release_id and the existing business grains.	Publication procedures insert only BUILDING generations and refuse writes after sealing. Cleanup is a separate authority.
published	Five authorized current views, five *_at_release functions, and a narrow release_status interface.	Deployment identity controls definitions and authorizations. Reader/BI has query access here only.
ops	Batch/run input records, control guards, runs, progress, leases/fences, releases, activation ledger, manifests, reader pins, and quality results.	Specific procedures control critical writes; workloads read only necessary metadata projections.
ops_*_api	Routine-only datasets for ingestion, transformation commits, orchestration, publication, and maintenance.	Invocation grants are separated by capability; runtime callers cannot edit definitions or read private ops tables.

This layout adds private state and release stores to the requested layers. Combining them into working datasets would reduce dataset count but weaken the boundary between disposable output and

confirmed decisions. Dataset-level separation is the recommended initial trade-off; table-level permissions could consolidate it later if automated denial tests prove equivalent isolation.

Partitioning clustering and logical versioning

Mechanism	Meaning	AgriDax use
Ingestion-time partitioning	Warehouse write/load date in _ PARTITIONTIME; it is not the producer event date.	Do not use it as the original receipt-time contract for replayed historical deliveries.
Receipt-column partitioning	DATE(ingested_at), where ingested_at is the platform's recorded delivery receipt timestamp.	Proposed RAW event/weather layout supports ingestion-window changed-key discovery and bounded recovery.
Business/event-time partitioning	An explicit event or business date column determines partitions.	Proposed daily and event release tables support date-filtered consumer reads.
Clustering	Ordered storage blocks within a table or partition; no uniqueness guarantee.	Candidate keys improve selective history/ release access only if measured query plans confirm pruning.
release_id versioning	Logical cohort identifier repeated across five mart tables.	Selects consumer generations; a STRING release ID is not itself a time partition or index.

Candidate recommendations, not benchmarks: RAW events can cluster by source_system, source_event_id, and field_id; weather by field_id and observed_at. Small master tables can remain unpartitioned; field state clustering by field_id and effective_from is conditional on measured access. Release daily facts can partition by business_date and cluster by release_id and field_id. Release events can partition by UTC event date and cluster by release_id, field_id, source_system, and source_event_id. Dimensions can remain unpartitioned with release/key clustering only when useful. Crop-cycle partitioning by planting date is optional and must be justified by observed consumer filters. [G8–G9]

Clustering is limited to four columns. Dynamic pointer predicates in authorized views may not prune as effectively as a literal release parameter; inspect query plans and bytes processed. Do not enable mandatory partition filters until global reconciliation, full rebuilds, and historical revision lookups supply correct bounds. An ingestion lookback cannot restrict the all-version lookup. Corrections that move a field or event date must update both old and new derived keys and partitions.

08 Incremental Processing and Winner Resolution

Immutable accepted batches

Each landing delivery uses a unique object name and a create-only write precondition. The landing writer receives objectCreator rather than overwrite/delete rights. Store the generation and checksum in the manifest. BigQuery Cloud Storage load URIs do not support generation suffixes: load the unique immutable name, verify its manifest before loading, and prevent overwrite by identity and lifecycle policy. Do not pass a gs:// object URI with a generation suffix and assume it pins the read. [G6–G7, G15]

Derive batch_id from the source identity, immutable object name, object generation, and documented contract version. Derive delivery_id from batch_id and row ordinal. Record the original receipt timestamp once. A retry of a failed load uses the same delivery identities; a separately delivered retry from the producer is a new physical batch, even when its content hash matches an earlier payload.

Use predictable BigQuery load job IDs and reconcile the existing job configuration and status after a duplicate or lost response. A new attempt ID is allowed only after a prior attempt is confirmed failed. The durable accepted-batch ledger, not the lifetime of a job ID, guarantees long-term idempotence. Structural failure keeps the original object and diagnostics but never accepts a partial batch. Row-level domain invalidity remains representable in RAW and quarantine. [G14]

Atomic acceptance and concurrent arrival

For each of the five known streams, a reviewed static acceptance procedure validates prepared row count, checksum identity, schema, and delivery-ID uniqueness. In one BigQuery transaction it mutates a seeded acceptance guard, verifies that the batch is not already accepted, inserts the complete RAW rows, and inserts the accepted manifest. A matching prior acceptance returns the prior result; conflicting evidence for the same batch is rejected. No BigQuery primary key enforces this protocol. [G1–G2]

Every acceptance transaction must mutate the same guard for the protected acceptance scope. Otherwise two transactions could both see no prior acceptance and append duplicates. Preparation and load jobs can run concurrently; only final acceptance is serialized initially. Conflict retries are bounded and use backoff. This small control bottleneck is a correctness choice to measure before increasing ingestion concurrency.

Frozen input and independent checkpoints

Within a control transaction, capture H_r and persist B_r, the batch IDs and manifest hashes visible in accepted_batches. Transformation and all global checks join this exact set. Ingestion accepts later batches independently. A batch whose acceptance transaction commits after capture is absent from B_r and appears in a later membership difference, even if accepted_at reflects an earlier transaction start.

For stream s, define $\Delta B(r,s)$ = eligible batches in B_r minus the batches represented by the selected confirmed state generation for s. Changed event and weather keys come from ΔB; resolve each key against all eligible deliveries in B_r. Do not derive progress from source_updated_at, event time, MAX(winner.ingested_at), or only the latest accepted timestamp. Invalid and non-winning deliveries still count as inspected progress.

The normal progress commit references a sealed canonical generation and its processed batch membership. A failed consumer release may leave this valid internal progress available. If a cache is removed or corrupted, reset or rebuild its referenced progress before processing again; skipping already marked batches without their state would lose decisions. Contract/code changes require cache compatibility validation or a full canonical rebuild.

Deterministic ranking and retries

Preserve event ranking by source_updated_at DESC NULLS LAST, ingested_at DESC, and physical delivery tie-breaker DESC. Preserve weather ranking at (field_id, observed_at) across all providers. Normalize source identifiers exactly as the current contract specifies. Source revision timestamps remain comparable within a logical key; producer behavior that breaks that contract is a source-quality risk, not something a cloud service repairs.

For historical migration, carry raw_record_id and its current ordering into the baseline. Proposed physical_tie_key: a legacy namespace plus a fixed-width raw_record_id representation, or a higher cloud namespace plus the stable delivery-ID hex value. Within the legacy namespace, preserve numeric ID ordering. New cloud exact ties use lexicographic delivery identity; they do not claim producer causality. Document the legacy/cloud boundary in an ADR and test it explicitly. Replaying the same frozen set must reproduce the same key and winner, without silently reordering migrated ties.

dbt-bigquery supports merge and partition replacement strategies. Start with changed-key MERGE into isolated working state seeded from the confirmed base, plus explicit uniqueness tests. A narrow destination time predicate can miss an older winner. insert_overwrite is appropriate only for complete affected partitions, including both sides of a moved correction. Initially rebuild the five marts from trusted canonical state as the repository does; optimize marts incrementally only after a correctness-preserving impact analysis. [G10]

09 Temporal Modeling and Snapshots

Effective business history

Current validated implementation. For each (field_id, effective_from), revisions choose one state. Business history orders those states and derives [effective_from, effective_to) intervals. Current fields select the greatest effective_from not beyond the run horizon. Crop pairing and historical field area use business-effective relationships. A correction can revise an old interval and alter derived facts. [R4, R8]

Target GCP architecture. Preserve those keys and half-open intervals with GoogleSQL window functions, using B_r for source eligibility and H_r for current-as-of logic. Explicitly specify UTC when deriving dates. Recompute neighboring intervals and all affected downstream keys when a historical state changes. BigQuery TIMESTAMP is used for instants; DATE represents UTC business dates; DATETIME is not a replacement for a timezone-aware instant.

Observed snapshot history

The existing snap_fields check strategy records changes in observed current field attributes, including source effective/revision timestamps, while excluding retry-only ingested_at changes. warehouse_observed_at records the warehouse observation instant. It is separate from business effective time and is not a complete bitemporal record of every intermediate source revision. [R8, G12]

The target preserves that meaning but does not let a dbt process write directly to shared confirmed snapshots. Build an attempt-specific proposed change set from the frozen current state and the committed snapshot head. A guarded commit checks the observation base, coordinator fence, and unique observation_id, then closes changed SCD rows, inserts new rows, and records the observation marker in one BigQuery DML transaction. Use actual warehouse observation time; do not substitute effective_from or the original event timestamp.

This controlled snapshot materialization is proposed implementation work. dbt's check semantics can compute changes, but ordinary dbt snapshot execution alone does not supply the cross-step fence and operational commit described here. Initially keep one observation coordinator. Retrying an already committed observation returns its marker; a stale-base observation is recomputed. Confirmed observed history may advance before consumer activation, preserving the current separation between warehouse observation and publication success.

RAW replay reconstructs effective history but cannot recreate observations that never occurred. Retain and back up observed snapshots and their commit ledger. A full business rebuild must not invent past observation timestamps, duplicate retry observations, or destroy confirmed historical SCD rows.

10 Quarantine and Data Quality

CURRENT — VALIDATED

Current validated implementation. The newest invalid delivery can become the canonical winner and remove a previously trusted row. Quarantine physically retains invalid deliveries; it is not merely a filter hiding them. A subsequent valid revision can restore trusted state. This distinction must survive both SQL migration and checkpoint advancement.

TARGET — NOT DEPLOYED

Target GCP architecture. Keep invalid winner state in state_store. Keep every structurally admitted invalid delivery in quarantine with original source values, delivery_id, batch_id, logical key, reason codes, contract version, and validation run references. Use controlled idempotent append by delivery identity; multiple runs may reference the same rejection without duplicating its physical evidence. Keep malformed whole-file evidence and parsing diagnostics in landing/ingest_work when no structurally valid row representation exists.

BigQuery type adaptation must preserve invalidity. PostgreSQL numeric values and casts do not have identical behavior to BigQuery NUMERIC. Preserve source literals, use explicit parsing such as SAFE_CAST, and distinguish missing, malformed, non-finite, and out-of-domain values. A failed parse must not become an unexplained NULL or zero. Validation must prevent such rows from trusted projections.

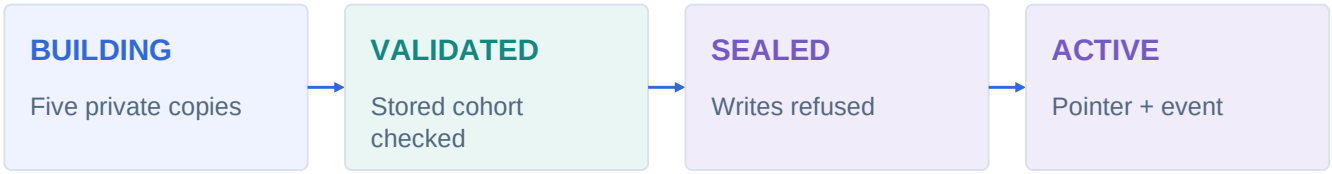
Required release gates

- Complete dbt DAG and artifact provenance for the run, attempt, image, contracts, input set, and invocation; reject contradictory case-normalized metadata keys.
- Business-grain uniqueness and not-null checks for all five marts; release_id-scoped relationships and schema compatibility with the published interface.
- All-history winner reconciliation inside B_r, including missed keys outside a bounded repair window, invalid winners, quarantine coverage, and trusted/rejected consistency.
- Temporal interval integrity, crop pairing boundaries, simultaneous planting rejection, historical field reference/name/status checks, and effective area semantics.
- UTC daily reconciliation, absent-versus-zero behavior, metric applicability, and cross-mart consistency.
- Freshness and future ingestion timestamp checks against frozen accepted evidence; refuse an automatic progress rewind. Declared freshness policy is not a measured operating SLA.
- Stored release row counts, schemas, content evidence, and aggregate reconciliations match the validated candidates before sealing and activation.

The current freshness configuration declares different event, weather, and master-data warning/error windows. Error gates block publication; warnings are recorded. Carry these as versioned policy defaults, then reassess them against real source cadence. Recent invalid retries can keep arrival freshness apparently healthy while trusted producer data is stale. Add trusted-data age and per-source arrival metrics rather than interpreting an arrival check as full source health. The cloud implementation should also close the supporting audit's historical field validator gap.

11 Publication and Release Strategy

Versioned releases are the preferred BigQuery pattern. They keep the expensive work outside consumer activation, maintain a complete prior generation, support explicit reader pins, and make rollback a control operation. BigQuery multi-statement transactions supply the short DML commit boundary; permanent tables and authorized views are provisioned beforehand because permanent DDL is not transactional. [G1]



Before activation: consumers retain the previous valid release.

A failed or abandoned generation is never made visible by partial materialization.

Figure 2. Release lifecycle. BUILDING, VALIDATED, SEALED, ACTIVE, SUPERSEDED, RETIRED, and FAILED are proposed controlled states; only activated retained generations are consumer-readable.

Five-mart publication contract

Existing consumer mart	Business grain within one release	Target physical and public interfaces
dim_farms	farm_id	release_store.dim_farms → published.dim_farms; dim_farms_at_release(rid)
dim_fields	field_id	release_store.dim_fields → published.dim_fields; dim_fields_at_release(rid)
fct_crop_cycles	crop_cycle_id	release_store.fct_crop_cycles → published.fct_crop_cycles; fct_crop_cycles_at_release(rid)
fct_field_daily_performance	field_id, business_date	release_store.fct_field_daily_performance → published.fct_field_daily_performance; corresponding *_at_release(rid)
fct_field_events	source_system, source_event_id	release_store.fct_field_events → published.fct_field_events; fct_field_events_at_release(rid)

Each candidate and stored mart is associated with run_id, attempt_id, and release_id; physical stored rows include release_id. Tests assert uniqueness on (release_id, existing business key) and relationships within that release. Authorized views expose explicit, stable consumer columns and the active release ID. Provenance additions and type changes are intentional contract migrations, not implicit SELECT * changes.

Preparation sealing and activation

Allocate the release_id in the run plan. Copy each of the five validated candidate outputs into precreated release_store tables. A controlled copy transaction checks the run/attempt and BUILDING status, mutates a release/control guard, inserts the release rows, and records its mart manifest. A repeated copy only returns success for matching evidence; it never appends another copy of an already materialized mart.

Validate the actual stored cohort after all copies: the five manifests, row counts, content evidence, schemas, uniqueness, relationships, and reconciliation gates must agree. Content evidence must use a documented canonical representation and collision-resistant digest, or exact row comparison where affordable; a commutative checksum alone is not proof of equality. The validated result is tied to sealed data, not only to a dbt success report.

Seal the generation with a controlled transaction. Every copy path must mutate the same protected release guard and refuse non-BUILDING status. Otherwise an earlier copy job could finish after sealing and invalidate the evidence. The dbt identity cannot write release_store; the publication identity cannot bypass procedures to rewrite sealed rows.

Activate with one short transaction: validate the live fence, the sealed release and five-mart manifest, and the expected previous activation epoch; compare-and-set the single ops.current_release row; assert exactly one updated row; insert the unique activation request event; update release status; commit. Concurrent activation conflicts are retried only after rereading the predecessor and eligibility. A stale run cannot automatically rewind a newer active release.

Initialize the control row once with a NULL release_id and activation_epoch 0. Before the first successful activation, release_status reports NO_ACTIVE_RELEASE; an empty view result must not be mistaken for an initialized valid release. The first activation uses the same complete five-mart gate as every later activation.

If the activation response is lost, inspect the BigQuery job and activation request ledger. A recorded activation is success even if the caller saw a timeout. A request already activated and subsequently superseded must not be reapplied. Rollback is a new authorized activation event pointing to a retained previously validated release; it does not rewind accepted batches, canonical progress, or observed snapshots.

Reader isolation and release pins

All five current views resolve the same control row inside a query. That preserves a single query’s cohort. Several independent queries can cross an activation boundary, so a report or BI refresh must first resolve one release_id through published.release_status and pass it to all five *_at_release functions. Filtering the current view by an old ID cannot retrieve an earlier generation.

Authorize the five table functions to access only retained generations that have an activation event and are not retired. This prevents a reader from selecting a sealed but never activated candidate by guessing its release ID. Readers have no direct release_store or ops access. Authorized table functions can expose filtered results without exposing base tables; the procedure/view authorizations and the chosen BI tool’s parameter support require cloud acceptance tests. [G4–G5]

For a long multi-query report, obtain and renew a reader pin through a narrowly authorized control routine. Validate the caller and release, bound its expiry by configured policy, and allow renewal/release only for its owner. Cleanup protects the pin until expiry. An expired client must fail explicitly or reacquire an available release; it must not silently switch to current. A BI connector without pin support has a per-query guarantee, not report-wide consistency.

The reader adapter checks pin validity and release status around its query sequence and aborts an expired or retired report. A function returning no rows is not sufficient evidence that a release exists: a valid mart can legitimately be empty. Include this distinction in the reader acceptance contract; do not promise that a parameterized SELECT alone enforces the client’s pin lifecycle.

Retention cleanup and cost

Proposed initial policy: always protect the active release and its two preceding activated releases; retain other activated releases for 30 days; retain failed or abandoned working generations for 7 days. These are configurable design defaults, not a demonstrated business requirement or RPO. Reader pins, canonical checkpoint references, and required snapshot/evidence references override cleanup. Original evidence and observed history require separate retention decisions.

Cleanup first marks an eligible generation RETIRED under the same activation/retention guard, checking that it is neither active nor pinned. Activation and new pins refuse retired generations. Delete its rows across the five private marts idempotently, then mark purge completion. A race between activation and retirement must conflict through a shared control mutation. A partially purged retired release remains unavailable; an active or protected release is never eligible.

Do not set automatic fact partition expiry by business date: the current release can contain old historical facts. Keep lineage, manifests, code/image references, and the activation ledger after row deletion. Purged releases may be rebuildable from retained RAW plus contracts, but they no longer offer immediate rollback, and effective replay does not recreate observed snapshot history.

Multiple releases repeat complete mart data and incur storage, copy, validation, and query costs. Measure compressed stored bytes, retained release count, and bytes scanned with real workloads. Release-date partitioning may simplify cleanup but can weaken business-date access; business-date partitioning with release clustering is the initial candidate. Shared snapshots/clones or a stable-table DML swap remain alternatives only if measured costs and reader requirements justify their additional constraints.

12 Orchestration with Cloud Workflows

TARGET — NOT DEPLOYED

Target GCP architecture. Define a versioned workflow with explicit normal, bounded-backfill, full-rebuild, resume, and controlled-rollback modes. Inputs include the requested code/image digest, contracts, bounds when applicable, and request_id. A scheduled trigger uses its own identity and OAuth authentication to invoke the Workflows execution API. Duplicate triggers create or resume the same logical request rather than assuming scheduler delivery is exactly once. [G17]

Workflow state	Required durable action	Exit condition
Plan and claim	Create/read run plan; claim coordinator with a new fence epoch; persist execution reference.	Arguments and code are compatible; another valid coordinator is refused or deferred.
Capture inputs	Persist B_r and H_r; record base canonical/snapshot heads and expected activation predecessor.	All input references are immutable and available.

Workflow state	Required durable action	Exit condition
Execute transform	Run the digest-pinned dbt job with run_id and attempt_id; keep the fence alive.	Execution and warehouse jobs are reconciled; canonical and observation commits are known.
Validate candidates	Load dbt artifacts; run full cohort, temporal, freshness, and global reconciliation gates.	Validation is tied to the current attempt and input digest.
Prepare publication	Run the publication job; copy five marts, validate stored rows, and seal.	All five stored manifests exist and immutable evidence agrees.
Activate and reconcile	Request compare-and-set activation; resolve timeout through job/event evidence.	The activation event identifies the committed result or a definite rejection.
Finalize	Record outcome, artifact references, and operational metrics; release coordinator conditionally.	Terminal status reflects internal progress and publication separately.

Workflows uses the official Cloud Run Jobs connector to execute and poll jobs. Its execution service account receives job execution permissions, not deployment permissions. For parameter overrides, use the specific Cloud Run Jobs Executor With Overrides role plus narrowly scoped read/poll permissions where required; do not grant Cloud Run Developer merely to pass a run ID. [G16, G18, G24]

Fencing and interrupted execution

Seed ops.pipeline_control with one coordinator record. Claim or renew it through a controlled transaction using owner_run_id, lease token, monotonic fence_epoch, and expiry. Critical progress, observation, release-copy/seal, and activation operations must validate the fence and mutate the protected control record in their transaction. A claim by a new owner then conflicts with a stale critical mutation rather than relying only on a snapshot read of the lease.

The protocol uses BigQuery’s documented transaction conflict behavior; it is not a general distributed lock service. Keep the guarded transactions bounded, measure their duration, and retry conflicts explicitly. Isolated output aliases are derived from immutable run/attempt IDs. The losing worker may finish an ordinary query in its own disposable namespace but cannot commit authoritative state or publication. Lease loss must be detected before starting additional work.

Cancellation of a Cloud Run execution is not proof that every submitted BigQuery job was cancelled. Record job IDs and locations, inspect their status, and cancel/reconcile owned jobs where possible. A resume workflow reads the batch, state, observation, release, and activation ledgers before deciding whether to retry. Never set FAILED and rerun promotion solely because the orchestrator disconnected.

Control SQL uses explicit rollback on exceptions. Permanent tables, view definitions, and routine authorizations are deployment operations outside data-run transactions. A failed transaction is retried as a new bounded operation after its result is known; the workflow does not blindly replay an arbitrary multi-step mutating request.

13 IAM and Security Model

CURRENT — VALIDATED

Current validated implementation. Separate PostgreSQL ingest, dbt, publish, reader, and administrative roles are present. The final grants restore dbt's CREATE rights on its schemas and publisher rights on published tables after administrative revocation. The mart post-hook supplies SELECT when dbt recreates candidates. Target GCP access must preserve these purposes through IAM rather than SQL username grants. [R7]

Use dedicated user-managed service accounts. All runtime identities use Application Default Credentials from the attached Cloud Run identity. Do not distribute GCP service-account keys or set GOOGLE_APPLICATION_CREDENTIALS to a key file in Cloud Run. BigQuery Job User belongs on the execution/billing project; dataset data permissions belong only on the required datasets or tables. [G3, G20]

Proposed identity	Minimum intended access	Explicitly excluded
ap-ingest	BigQuery Job User; Data Editor on ingest_work; Object Viewer on landing; invocation access to acceptance routines; Object Creator on landing only if it also collects sources.	Direct RAW update/delete, transformation writes, release writes, IAM administration.
ap-transform	Job User; Data Viewer on RAW, selected state/ snapshot tables and run metadata; Data Editor on working staging/intermediate/candidate; invocation of controlled state/observation commits.	Direct committed-state, quarantine, snapshot, release_ store, or current-release mutation.
ap-publish	Job User; read the five candidate outputs and validation projections; invoke copy/seal/activate routines.	Read all analytics by default, arbitrary RAW access, direct sealed-generation DML, role administration.
ap-reader / BI group	Job User on a designated query project; Data Viewer on published views/functions; invoke limited pin operations.	Candidate, RAW, quarantine, ops, state_store, and release_ store base access.
ap-orchestrate	Execute configured jobs with permitted overrides; required job/operation reads; invoke plan/fence/ resume control routines.	Change job definitions, impersonate workload identities, edit consumer data, read external secrets.
ap-schedule	Workflows Invoker for the intended workflow, authenticated scheduled invocation.	Warehouse access or Cloud Run deployment.
ap-maintain	Invoke guarded cleanup, reconciliation, and retained-evidence maintenance routines.	Activate a release or rewrite accepted source evidence.

Proposed identity	Minimum intended access	Explicitly excluded
ap-deploy	Reviewed deployment permissions on the project resources, BigQuery definitions and authorizations, workload service-account attachment, and image publication.	Routine pipeline execution with an unreviewed universal credential bundle.

Controlled mutation boundaries

BigQuery Data Editor includes destructive data operations; granting it on RAW would not reproduce PostgreSQL's INSERT-only ingestion privilege. Runtime callers instead execute static authorized stored procedures. Those procedures can act beyond the caller's ordinary table permissions, so their definitions and authorizations are security-sensitive deployment assets. Never accept arbitrary SQL, table paths, or unvalidated dynamic identifiers as caller parameters. [G3, G11]

Use separate routine-only API datasets for ingestion, transformation commits, orchestration, publication, and maintenance capabilities. The initial proposed names are ops_ingest_api, ops_transform_api, ops_control_api, ops_publish_api, and ops_maintain_api. Grant only the required routines.get capability through a narrow custom role or Data Viewer on the dataset containing that identity's callable routines. Keep ops tables private. published contains only approved reader views, functions, and pin interfaces. Proven resource-scoped invocation permissions could consolidate these API datasets later.

Routine authorization must be re-established and denial-tested when CREATE OR REPLACE changes a routine. Runtime identities must not create or replace authorized procedures. The deployment identity owns the definitions and dataset authorization entries. Use the established authorized-routine mechanism; newly documented beta routine roles are not a dependency of this design. [G11]

Publisher access narrows the audit's broad local analytics SELECT grant to five candidates and required evidence. dbt's PostgreSQL SELECT post-hook is removed in the BigQuery adapter path; deployment-managed IAM and authorized interfaces replace it. Recreating a working relation cannot grant a reader new access. Test direct DML denials, dataset access denials, procedure scope, and source object overwrite denial with the actual workload identities.

Assume a private project, uniform bucket-level access, blocked public bucket access, no public datasets, and least-privilege human access. Google-managed encryption is the default proposal. Customer-managed keys, VPC Service Controls, tenant isolation, and row/column policy controls require a demonstrated governance or confidentiality requirement; they are not asserted as already deployed protections.

14 Secrets Management

TARGET — NOT DEPLOYED

Target GCP architecture. The current source fixtures do not demonstrate an external API credential requirement. Do not create fictitious secrets. If an implemented collector later needs an API token, place it in Secret Manager, grant Secret Accessor on that secret only to the collector identity, pin or explicitly record the chosen version, and rotate through a tested deployment/run procedure. [G19]

GCP access is an identity concern, not a Secret Manager key-storage pattern. The dbt BigQuery profile uses keyless ADC/OAuth credentials in the job. Configuration such as project, location, dataset names, bounds, and image digest is non-secret versioned configuration. Source tokens, connection credentials, and sensitive producer headers are never placed in dbt artifacts, Workflow arguments, plaintext logs, landing manifests, or committed .env files. [D1, G20]

Run artifacts retain credential-free profiles and configuration digests rather than secret values. Secret Manager auditability supplements application records; it does not replace source provenance. A secret retrieval or authorization failure fails the relevant job before accepting an unverifiable source delivery.

15 Observability and Monitoring

CURRENT — VALIDATED

Current validated implementation. Operational runs, checkpoints, fingerprints, dbt artifacts, and acceptance outputs provide evidence. The repository does not demonstrate a deployed monitoring service or production incident response history. Target instrumentation builds on those records rather than claiming an existing cloud operating model.

Emit structured JSON logs to stdout/stderr with run_id, attempt_id, release_id, batch_id where applicable, workflow execution reference, stage, job ID/location, image digest, and terminal error category. Cloud Run collects these into Cloud Logging. Exclude payloads and secrets; record safe diagnostic reason codes and references to restricted evidence. [G21]

Signal	Purpose	Proposed response
Unaccepted landing batch age / failed load	Identify structural input failures and incomplete batch preparation.	Retain evidence and alert an operator; do not accept partial rows.
Trusted-data and accepted-arrival freshness	Separate source health from recent invalid/retried arrivals.	Warn or block according to versioned quality policy.
Canonical invalid winners / quarantine reasons	Detect loss of trusted coverage and contract drift.	Inspect producer changes and compare against the previous valid release.
Coordinator expiry / fence rejection / orphan jobs	Identify interrupted or stale processing.	Reconcile ledgers and warehouse jobs before retry.
Release age / failed gates / incomplete copies	Detect a stale consumer release without exposing bad candidates.	Keep the current release; report the failing gate and candidate lineage.
Activation mismatch / unknown commit / cleanup race	Detect unresolved visibility decisions.	Inspect request/job evidence; refuse a speculative pointer update.
Bytes processed / slot time / storage by layer	Measure optimization and retention cost.	Adjust bounds, layout, cadence, and retention after correctness checks.

Create Monitoring dashboards and alert policies for these operational conditions and job failures. Thresholds, notifications, refresh cadence, and escalation ownership are proposed decisions to agree after measurements. Budget alerts support cost visibility; they do not enforce a workload SLA or guarantee a hard spending cap. Cloud logs are not the only copy of provenance: persist dbt artifacts and validation reports under immutable run/attempt object paths and reference their hashes from ops. [G25]

16 Recovery and Backfill Strategy

CURRENT — VALIDATED

Current validated implementation. Bounded backfill uses a half-open ingestion interval [start, end). It requires an initialized normal processing baseline, does not advance the normal checkpoint, and still executes complete candidate validation and global winner reconciliation. Combining bounded recovery with a full refresh is rejected. The target preserves these rules using batch membership and confirmed state rather than a scalar PostgreSQL checkpoint. [R6]

Target backfill plans bind a frozen B_r, bounds on recorded ingested_at, source/contract versions, code digest, and an initialized canonical base. Select affected keys from deliveries in the bounds, then resolve their entire eligible revision history in B_r. Global reconciliation prevents a bounded repair from publishing a cohort that misses unrelated accepted keys. If that gate detects broader missing work, run normal catch-up or an explicit full rebuild; do not widen the window silently.

Normal catch-up commits new processed batch membership with its confirmed canonical state. Bounded backfill does not mark unrelated batches as processed or advance normal progress. A full rebuild computes all eligible canonical state from a frozen input set and creates a new state generation. Never truncate shared RAW, active release rows, or observed snapshots to make a rebuild convenient.

Recovery sequence

1. Identify the last confirmed batch, canonical, observation, and activation decisions. Inspect job results for any ambiguous operation.
2. Choose resume, normal catch-up, bounded repair, or full rebuild. Record a new request/attempt and preserve the original evidence.
3. Reclaim the coordinator with a new fence; prevent the old worker from committing. Keep its working outputs private until reconciled.
4. Rebuild only disposable state or create a new confirmed generation; validate full canonical and consumer contracts.
5. Activate only a sealed valid release. If needed, roll back through a new activation event to a retained release without rewinding ingestion progress.
6. Verify consumer pins, archive failure evidence, and retire unreferenced working outputs through guarded maintenance.

Choose backup, object retention, and disaster recovery policies only after business requirements are agreed. BigQuery’s built-in recovery features are useful operational tools, but they do not replace application release retention or preserved observed history. RPO and RTO are intentionally unspecified because the repository supplies no measured requirement or drill evidence.

17 Failure Scenarios and Expected Behavior

Failure or correction	Authoritative behavior	Consumer result and recovery
Ingestion accepted; transformation fails	Accepted RAW and batch manifest remain durable. Only confirmed component progress may advance.	Previous active release remains visible; a later run includes unprocessed accepted batches.
dbt build fails	Keep artifacts and private attempt outputs. Do not validate/seal a failed cohort.	No activation. Resume/retry with an isolated attempt; previously committed observations remain observations.
Validation fails	Record gate failure and input/code lineage. Release is not activation eligible.	Previous release remains; correct source/code or choose a controlled repair.
A mart copy or publication preparation fails	Incomplete generation stays BUILDING/FAILED and private. Confirm each copy marker before retry.	No partial mart visibility. Finish valid copies or abandon the generation.
Activation fails or response is lost	Reconcile BigQuery job and activation request ledger; do not equate timeout with rollback.	Pointer is either committed to the complete release or remains prior. Retry only the unresolved operation.
Orchestration is interrupted	Durable plan and control ledgers survive. Fence expiry allows controlled recovery; orphan jobs are reconciled.	Readers continue on current. A stale worker cannot commit state or activate.
Same batch is retried	Stable IDs, load-job reconciliation, and acceptance guard return existing acceptance.	No duplicate accepted rows or repeated progress; canonical output is unchanged.
Producer redelivers identical content	New physical delivery is retained as a new batch; deterministic ordering applies.	It does not fabricate new observed history merely because ingested_at changes.
Late valid or invalid correction	Next frozen input includes its batch. Changed-key resolution reads all versions and recomputes downstream impact.	New valid release can revise past facts; invalid latest state removes trusted data only after successful cohort validation.
New generation partly materialized but never activated	It remains private and cannot be queried through reader functions because it lacks an activation event.	Current release remains visible; cleanup later retires it safely.
Reader spans activation	Current views alone may observe different generations across separate queries.	A pinned report reads one retained activated release; an expired pin fails explicitly.

Failure or correction	Authoritative behavior	Consumer result and recovery
Cleanup races with activation or a pin	Shared guard and protected-state checks prevent conflicting eligibility decisions.	An active/pinned release is not purged; retired partially deleted generations remain unreadable.
Committed cache or snapshot evidence is missing	Progress references are treated as invalid; rebuild effective state or restore observed evidence.	Do not skip batches on a missing cache or invent past observations; active retained release continues.

18 Scalability and Cost Considerations

The repository provides fixture-scale correctness evidence, not a validated production workload profile. Start with native BigQuery batch loads, one transformation coordinator, bounded job retries, and on-demand query measurement. Select reservations, job parallelism, schedules, partition layouts, and retention only after observing real data and query patterns.

Changed-key discovery reduces the keys requiring revision decisions, but their historical versions still need scanning. Master and effective-state dependencies can cause broad recomputation. The five marts initially remain complete generation builds. Measure the changed-key fraction, history depth, affected business dates, dbt destination scans, stored-release validation scans, and retained bytes before claiming the pipeline is cheaply incremental.

Partitioning by ingestion can reduce changed-key selection but does not make business-history joins free. Clustering by logical key can help all-version reads, yet its benefit depends on data size and access selectivity. Release copies duplicate complete data; retaining many small releases and retaining a few large releases have different economics. No dollar estimate or throughput figure is supported by the baseline.

Use dry runs where applicable, per-query maximum_bytes_billed, job labels, and post-run actual bytes/slot metrics. Dry-run estimates for clustered tables can be upper bounds; per-query limits are not an end-to-end budget. Global reconciliation queries must remain complete even if they are expensive. Adjust implementation or cadence rather than silently dropping validation to satisfy a cost target. [G13]

Measure acceptance-guard and coordinator contention before increasing concurrency. Any broader concurrency scope requires a new ADR and duplicate-acceptance and fence tests.

19 PostgreSQL-to-GCP Mapping Table

Current capabilities and target mechanisms differ deliberately. This mapping preserves application contracts while avoiding assumptions that PostgreSQL locks, grants, indexes, or transactional DDL exist unchanged in BigQuery.

PostgreSQL / local capability	GCP implementation	Preserved invariant	Design difference
Append-only RAW tables and physical raw_record_id	Immutable landing objects; accepted batches; stable delivery IDs in BigQuery RAW.	Physical evidence and deterministic replay.	GCS identity plus a guarded ledger replaces database-generated delivery identity for new cloud input.
INSERT-only ingestion role	Ingest work dataset and static authorized acceptance procedures.	Runtime cannot rewrite accepted evidence.	BigQuery updateData is broader than INSERT; IAM alone does not provide the same DML distinction.
Advisory writer and ingestion locks	Fenced processing coordinator plus independently accepted batches.	One authoritative processing commit; stable input while ingestion continues.	Persist batch membership and isolate attempts; no PostgreSQL session-lock emulation.
REPEATABLE READ processing/publication	Frozen B_r for the whole run; BigQuery snapshot-isolated control transactions.	Consistent eligible source set and coordinated visibility.	A long dbt DAG is not one warehouse transaction; immutability extends consistency across jobs.
Ingestion-time checkpoint and lookback	Processed batch membership linked to sealed canonical state; receipt bounds for repair.	Progress independent of winners and retry ordering.	Membership discovers late acceptance even with an older timestamp.
Incremental winner tables	Run-scoped changed-key resolution over all versions; confirmed state_store generation.	Deterministic trusted/invalid canonical decisions.	Explicit GoogleSQL MERGE and uniqueness gates replace reliance on enforced keys.
Effective field history and snap_fields	GoogleSQL intervals plus controlled observed SCD commits.	Business time differs from warehouse observation time.	Snapshots require a fenced, idempotent materialization boundary.

PostgreSQL / local capability	GCP implementation	Preserved invariant	Design difference
Physical rejected-delivery tables	Private quarantine dataset with delivery lineage and reason codes.	Invalid evidence is retained and can become canonical.	Parsing/source literals are explicit; type conversion must not erase invalidity.
analytics candidate marts	Run/attempt candidate relations for the same five marts.	Failed transformation data is private.	dbt never writes the consumer release store.
Transactional DELETE + INSERT of five published tables	Versioned release_store; validation/seal; one current_release transaction.	All five marts become visible as one valid generation.	Stable views replace stable physical published table identity; preparation is outside activation.
Reader transaction pins an earlier table snapshot	Resolve release_id once; authorized *_at_release functions and reader pins.	Reports can keep using a previous valid cohort.	Pinning is an explicit report/API contract across independent BigQuery queries.
Administrative grants and mart SELECT post-hook	Deployment-managed IAM, routine/view authorizations, candidate read scopes.	Dedicated identities and rerunnable access provisioning.	No username grant hook in BigQuery; routine replacement reauthorization is tested.
dbt manifests, run_results, validation fingerprints	Immutable artifact objects; ops provenance, job IDs, input and mart manifests.	Evidence identifies the data, code, attempt, and activation.	An image digest and accepted input set supplement dbt metadata.
PowerShell / Docker Compose local execution	Digest-pinned Cloud Run Jobs and a versioned Workflow; Scheduler when justified.	Reproducible ordered execution.	Compose remains the local reference path; cloud retries are reconciled from durable state.
PostgreSQL indexes and enforced constraints	Partition/clustering candidates; dbt and stored-cohort uniqueness/reference gates.	Correct grains and relationships.	Clustering is not an index, and BigQuery PK/FK declarations are not enforcement.

20 Trade-offs and Explicit Non-Goals

The target favors correctness, auditable batch boundaries, and a modest service set. The cost is additional metadata and controlled procedures, complete release copies, explicit reader pins, and adapter work beyond a mechanical dbt profile change. These are justified by existing AgriDax guarantees, not by a claim that every data platform needs this design.

Decision	Trade-off	Non-goal or limitation
Immutable native RAW plus object evidence	Storage is duplicated across original bytes and queryable rows.	No streaming ingestion or row-level CDC guarantee is introduced.
One processing coordinator; concurrent ingestion	Simplifies observed history and authoritative commits; may bound throughput.	No measured multi-writer production claim.
Versioned complete five-mart releases	Clear rollback and cohort visibility; storage/copy cost grows with retention.	No permanent transactional DDL or opaque multi-table swap.
Pinned reader functions	Protect report-wide consistency if the client follows the contract.	Unpinned independent queries are not guaranteed one release.
Protected state and mutation routines	More deployment assets and denial tests; stronger separation of duties.	No claim of immutable protection against project administrators.
Complete domain contracts	Preserves the current metric meanings and limitations.	No inferred sustainability validity, calendar completeness, geospatial normalization, or planted-area yield.
Measured optimization	Keeps cost/scale decisions evidence based.	No invented SLA, RPO/RTO, savings, throughput, or production GCP experience.

Authorized procedures are a trusted security boundary, not protection from a malicious administrator who can change definitions or permissions. Source systems must still supply comparable revisions and valid keys. Object evidence is replayable only while retained and readable; code, schemas, and dependency versions must also remain available. These are explicit operational dependencies.

Terraform, cloud CI/CD, cloud acceptance, connector credentials, regional selection, retention requirements, BI pin compatibility, and performance measurements are implementation decisions still to be completed. BigQuery time travel, materialized views, table clones, row-level policies, and additional services may be evaluated later for a concrete problem. None is assumed deployed or required merely to make the target look more complex.

21 Migration Roadmap

Migrate by proving equivalence before moving consumer access. Keep the PostgreSQL implementation as the reference for deterministic fixture runs and the documented domain semantics. Use synthetic and retained source evidence rather than production-scale claims.

1. Pin the repository baseline, copy the complete source contracts and acceptance scenarios, and document cloud-specific delivery identity, release pinning, and retention ADRs.
2. Translate source types and SQL incrementally. Compare canonical winners, invalid state, quarantine, field intervals, snapshot change semantics, and all five mart outputs against equivalent PostgreSQL input sets.
3. Implement batch acceptance and frozen run inputs before relying on changed-key processing. Demonstrate late acceptance and ingestion during transformation.
4. Implement protected canonical/snapshot commits and the complete five-mart release mechanism. Fault-inject between every preparation, seal, and activation boundary.
5. Exercise actual ingestion, transform, publisher, reader, and deployment identities. Validate both allowed paths and direct-access denials after redeployment.
6. Run the scheduled/resume workflows and measure cost/query behavior. Keep cloud results private until the full acceptance gate passes.
7. Move consumer reads through the documented authorized interface, validate multi-query pin behavior, and retain an explicit rollback path for the agreed release window.

Do not remove the local reference path when introducing a BigQuery target. The repo’s runtime pins and dbt-postgres constraints are evidence for the current environment, not a tested dbt-bigquery lock. Build a separate compatible cloud lock on the selected Python/dbt/adapter versions and record that exact runtime in acceptance evidence. The cloud image must include the dbt/SQL resources currently expected in the working tree.

22 Future Terraform Implementation

TARGET — NOT DEPLOYED

Target GCP architecture. Terraform should express reproducible environments after the schema, security, and release contracts are stable. It is a proposed implementation deliverable; the inspected repository contains no Terraform deployment. The Google provider manages declared resources and state, but Terraform does not implement the runtime winner, batch, or release algorithms. [G23]

Proposed module boundaries

- foundation: configured project and locations, required APIs, labels, service accounts, and a restricted remote state bucket.
- storage: landing and artifact buckets, create/read identities, retention/lifecycle policies, and public-access protection.

- warehouse: the datasets, explicit base schemas, authorized views/functions/procedures, and protected metadata initialization.
- security: dataset/table/routine access, authorized dataset entries, workload role bindings, and Secret Manager references only when needed.
- execution: digest-pinned Cloud Run job definitions, Workflow source, Scheduler configuration, retries/timeouts, and identity attachment.
- observability: dashboards, alert policies, log configuration, and cost monitoring based on agreed operational ownership.

Keep SQL contracts and procedure/view definitions as versioned deployment inputs. Treat authorization after routine replacement as an explicit dependency and acceptance check. Use additive compatibility migrations for consumer schemas; do not allow a dataset destroy or view rewrite to bypass release validation.

The state backend must be restricted because Terraform state can contain sensitive configuration. Keep secret values outside state where possible. Separate reviewed plan/apply authority from runtime workloads, protect evidence datasets from accidental deletion, and pin provider versions. Use GitHub Actions with short-lived Workload Identity Federation for a future pipeline rather than committing a service-account JSON key. [G22]

Cloud CI/CD should run format/static checks, Python tests, dbt parsing/compilation, SQL/contract checks, dependency checks, image build validation, Terraform validation/plan, and cloud acceptance in an isolated environment. A reviewed deployment changes infrastructure definitions; a normal data run changes accepted batches and releases through runtime protocols. The current repository does not implement this cloud pipeline.

23 Architecture Decision Summary

Decision	Selected approach	Alternatives and reason
AD-01 Evidence	Immutable GCS objects plus native accepted RAW batches.	Mutable external sources lose run stability; content-only deduplication loses distinct physical evidence.
AD-02 Run boundary	Persist exact accepted batch IDs and a UTC horizon.	Timestamp-only checkpoints can skip late acceptance; a long cross-job transaction is not the execution model.
AD-03 Canonical progress	Seal state generations and processed membership independently of release activation.	Winner-derived checkpoints skip invalid/non-winning deliveries; consumer pointers are not processing checkpoints.
AD-04 Concurrency	One fenced transformation coordinator with isolated attempts; ingestion remains independent.	Scheduler exclusivity and task count are insufficient; PostgreSQL advisory locks do not transfer.

Decision	Selected approach	Alternatives and reason
AD-05 Temporal state	Business intervals plus fenced observed SCD commits.	Source effective timestamps cannot recreate warehouse observations; unguarded shared snapshot writes allow stale workers.
AD-06 Publication	Complete versioned releases, protected sealing, one compare-and-set pointer.	Stable-table DML replacement is possible but retains larger mutation work at activation and complicates old-release reads.
AD-07 Readers	Authorized current views plus explicit release-pinned functions.	Independent current-view queries can cross activation; direct base access exposes incomplete generations.
AD-08 Privileges	Dedicated identities and narrow authorized mutation routines.	RAW Data Editor and universal credentials violate the intended mutation boundary; PostgreSQL grants/hooks are not portable.
AD-09 Retention	Protect active/prior releases and reader pins; guarded retirement and idempotent purge.	Blind partition expiry can delete active historical facts; unlimited complete releases have unbounded storage cost.
AD-10 Execution	Cloud Run Jobs, Workflows, and justified scheduling.	No demonstrated need for Composer, Dataflow, Pub/Sub, Dataproc, or GKE.
AD-11 Physical layout	Receipt/business partition candidates and key/release clustering, tested by measurement.	Clustering is not uniqueness or indexing; overbroad time pruning breaks revision correctness.
AD-12 Reproducibility	Code/image/contract locks, frozen inputs, durable job/evidence references, future Terraform.	Latest tags, mutable input sets, and retry-by-assumption do not identify a reproducible execution.

These are proposed decisions derived from AgriDax’s implementation and supported by current official capabilities. They become implemented decisions only after code, permissions, cloud acceptance, and measured operational behavior demonstrate them. Alternative designs remain valid if they preserve the same invariants with clearly superior measured simplicity or cost.

24 Implementation Roadmap

Phase 1 — GCP foundation

Define project/environment separation, one compatible location, bucket evidence policy, Artifact Registry, dedicated identities, and keyless execution. Establish source and consumer data classification before selecting region or advanced security controls. Exit gate: a pinned test image runs with only its intended identity, and landing objects cannot be overwritten by the producer/collector.

Phase 2 — BigQuery schemas and contracts

Create raw, ingest_work, analytics_staging, analytics_intermediate, state_store, quarantine, snapshots, candidate, release_store, published, ops, and routine API boundaries. Define types, delivery/ranking contracts, guards, control rows, and evidence schemas. Avoid unenforced-key assumptions. Exit gate: contract fixtures preserve invalid literals, UTC semantics, grains, and receipt history.

Phase 3 — ingestion

Implement immutable source/manifest writes, deterministic batch/delivery IDs, explicit load jobs, structural diagnostics, atomic guarded acceptance, and retry reconciliation. Exit gate: concurrent attempts accept one physical batch exactly once, new physical retries remain evidence, and interrupted acceptance has a provable result.

Phase 4 — dbt-bigquery adaptation

Replace PostgreSQL-specific casts, timestamp handling, FILTER/bool_or/LATERAL usage, duration calculations, JSON behavior, indexes/grants, and snapshot execution as needed. Preserve changed-key all-version resolution, invalid winners, field intervals, metric semantics, and physical quarantine. Implement controlled canonical/snapshot commits. Exit gate: equivalent frozen inputs match the local reference and historical validation gaps are covered.

Phase 5 — publication/release mechanism

Implement candidate manifests, controlled five-mart copies, stored-cohort validation, sealing, compare-and-set activation, rollback, authorized views, pinned functions, pin retention, and guarded cleanup. Exit gate: no injected copy/validation/activation failure exposes a partial release, and a pinned reader remains on the old generation during a successful new activation.

Phase 6 — orchestration and observability

Implement versioned Workflow modes, coordinator fences, job polling/reconciliation, durable resume decisions, structured logs, evidence storage, dashboards, and alerts. Configure Scheduler only after cadence is agreed. Exit gate: interrupted orchestration resumes without duplicate acceptance, observation, or activation, and stale jobs cannot commit.

Phase 7 — Terraform and CI/CD

Codify the validated resource/permission model, authorized interfaces, lifecycle policies, image digests, Workflows, monitoring, and short-lived CI identity. Keep plan/apply separate from data-run authority. Exit gate: repeated deployment preserves access boundaries and existing release/evidence data, including routine reauthorization.

Phase 8 — validation and cloud acceptance testing

Port native scenarios and add cloud-specific concurrency, late-acceptance, lost-response, fence, snapshot-base, schema, retention, and authorization tests. Measure query/storage costs and recovery behavior before setting operating targets. Exit gate: complete cloud acceptance evidence names the commit, image, runtime, identities, region, input set, job IDs, and all results; the five marts and consumer pins satisfy the documented contracts.

Acceptance matrix for the target

Category	Minimum cloud demonstrations
Batch evidence and replay	Same-batch retry; distinct producer retry; checksum mismatch; structural failure; duplicate concurrent acceptance; immutable object overwrite denial.
Processing and temporal state	Late acceptance older than capture timestamp; ingestion during build; all-version late correction; invalid latest winner; bounded repair; missing cache; effective/observed separation.
Snapshots and progress	Retry after observation commit; stale observation base; component progress without consumer activation; no invented observation history after rebuild.
Publication and readers	Failure after each mart copy; stored-data drift; seal race; competing/stale activation; unknown response; rollback; old-reader pin and new-reader visibility.
Access and reproducibility	Each workload's permitted path and negative direct DML/read tests; routine replacement authorization; digest/contract drift rejection; credential-free artifacts.
Maintenance and operations	Pin/activation versus cleanup races; orphan BigQuery jobs; workflow interruption/resume; measured scan/storage behavior; evidence retention and restoration.

25 Architecture Design Walkthrough

This section summarizes the architecture from the validated local implementation through the proposed GCP target design, highlighting the main design decisions, preserved invariants, and trade-offs.

Platform and Validated Baseline

AgriDax is a batch data platform for agricultural master data, events, weather, and sustainability records. The current implementation uses Python, PostgreSQL, and dbt-postgres. It has a native acceptance result of 142 passing checks in the supplied evidence. Its main concern is deciding which source revision is authoritative, preserving invalid deliveries for investigation, and publishing five consumer marts as one valid cohort.

The repository already separates RAW evidence, canonical state, effective field history, observed snapshots, candidates, and published tables. Those boundaries are preserved in the proposed GCP migration. The cloud design documented here is a target architecture and has not yet been deployed or benchmarked.

Ingestion and deterministic decisions

The target design begins with immutable files in Cloud Storage because a data correction must not erase evidence that was delivered previously. A batch manifest identifies the object generation and checksum. The ingestion job prepares rows and then requests a guarded BigQuery transaction that writes the RAW deliveries and the accepted-batch record together. Retrying the same batch returns its existing acceptance, while a separate delivery of identical content remains distinct physical evidence.

A processing run persists the exact accepted batches that form its input set. Ingestion may continue while dbt executes, but later batches do not alter the input of the active run. Progress records which batches have been inspected independently of which row became the canonical winner. This distinction is important when a delivery is invalid or does not change the current winner.

Changed-key processing identifies affected event or weather keys and then resolves all eligible historical revisions for those keys. It does not select only the newest row within a recent processing window.

The newest invalid revision remains the canonical decision and removes the older trusted value from the trusted projection. The invalid delivery is physically retained in quarantine together with its delivery evidence and rejection reason. A later valid correction can restore the key. Falling back automatically to an older valid revision would hide the current invalid source state and is therefore intentionally prevented.

Temporal state and publication

The platform maintains two distinct temporal perspectives. Effective field history describes when a state applies in the business domain, while observed snapshots describe when the warehouse observed a change in the current field state.

Replaying retained source files can reconstruct effective history, but it cannot recreate warehouse observations that never occurred. The target cloud design therefore commits observed changes idempotently and preserves that history separately from business-effective state.

Publication uses versioned releases in BigQuery. Each release contains:

- dim_farms
- dim_fields
- fct_crop_cycles
- fct_field_daily_performance
- fct_field_events

The five marts are copied into private release tables, validated as stored data, and sealed before they become eligible for activation. Only after those checks succeed does a short transaction update the single current-release pointer.

If dbt execution, validation, or release preparation fails, the previously active release remains visible to consumers.

A current-release pointer alone is not sufficient for a report that executes multiple independent queries. The report therefore resolves one release_id and uses authorized functions to read all five marts from that same generation. Cleanup protects active releases and valid reader pins.

This preserves the practical ability of a reader to remain on an earlier valid release without assuming that BigQuery provides the same session-level snapshot behavior as PostgreSQL.

Orchestration identities and trade-offs

Cloud Run Jobs are used for the existing batch-oriented Python and dbt workloads. Cloud Workflows coordinates the execution stages and reconciles interruptions using durable operational evidence.

One fenced transformation coordinator is responsible for committing authoritative state. An abandoned or superseded worker may complete work only within its isolated attempt-specific outputs and cannot commit authoritative state or activate a release. Ingestion remains independent from transformation execution.

Composer, Dataflow, Pub/Sub, Dataproc, and GKE are intentionally excluded from the initial design because the current requirements do not demonstrate a need for those services. Additional platform components should be introduced only when a concrete workload or operational requirement justifies them.

Ingestion, transformation, publication, reader, orchestration, maintenance, and deployment responsibilities remain separated through dedicated identities. Runtime access uses attached service accounts rather than stored service-account keys.

Because BigQuery table-write permissions are broader than PostgreSQL INSERT-only privileges, protected mutations are performed through reviewed authorized procedures and controlled interfaces.

The publication identity cannot directly rewrite sealed releases, and readers cannot access RAW, candidate, quarantine, operational, or private release data.

The principal trade-off is additional control metadata, procedure-level validation, operational evidence, and storage for retained complete releases. These costs provide stronger release isolation, deterministic recovery, and auditable state transitions.

Partitioning and clustering remain design hypotheses tied to ingestion windows, event dates, business dates, and logical keys. They are not treated as validated performance optimizations until measured against real workloads.

Cloud acceptance testing must cover late corrections, duplicate deliveries, stale workers, interrupted orchestration, ambiguous activation responses, reader pins, release cleanup, and access-denial scenarios before performance, scalability, or availability claims are made.

26 Official References

Primary documentation reviewed on 13 October 2026. References justify material service capabilities; the application protocols, dataset layout, retention defaults, and roadmap remain proposed design decisions. Links point to official Google Cloud or dbt documentation rather than third-party reference architectures.

[G1] BigQuery multi-statement transactions

Snapshot isolation, atomic DML, conflict behavior, and permanent-DDL restrictions.

[G2] BigQuery primary and foreign keys

Constraints are not enforced; release quality gates remain necessary.

[G3] BigQuery IAM roles and permissions

Project job permissions versus dataset/table data permissions.

[G4] BigQuery authorized views

Private base data, public interfaces, and location compatibility.

[G5] BigQuery table functions

Parameterized, authorized reader interfaces for retained release IDs.

[G6] Load Parquet data from Cloud Storage

Cloud Storage load URI restrictions, including object generation suffixes.

[G7] Cloud Storage request preconditions

Create-only object writes with generation-match preconditions.

[G8] BigQuery partitioned tables

Ingestion-time and column-based partition semantics.

[G9] BigQuery clustered tables

Block pruning, partition interaction, and the four-column clustering limit.

[G10] dbt BigQuery configurations

MERGE, partition replacement, and adapter-specific materialization configuration.

[G11] BigQuery authorized routines

Controlled mutation capabilities, caller access, and reauthorization after replacement.

[G12] dbt snapshot strategy

Check-based observed change detection and configurable snapshot semantics.

[G13] BigQuery cost best practices

Query estimates, maximum bytes billed, and measurement-based cost control.

[G14] BigQuery reliability

Predictable load job IDs, ambiguous responses, and retry reconciliation.

[G15] Cloud Storage IAM roles

Object Creator prevents overwrite/delete; Object Viewer scopes retrieval.

[G16] Execute Cloud Run jobs

Job execution, completion, overrides, and operational interfaces.

[G17] Schedule Workflows with Cloud Scheduler

Authenticated scheduled execution with a dedicated trigger identity.

[G18] Cloud Run IAM roles

Jobs Executor, Jobs Executor With Overrides, and separate deployment authority.

[G19] Secret Manager best practices

Scoped secret access, identity-based authentication, versions, and rotation.

[G20] Cloud Run service identity

User-managed identities and keyless Application Default Credentials.

[G21] Cloud Run logging

Structured container logs and job log collection.

[G22] Workload Identity Federation for deployment pipelines

Short-lived external CI authentication instead of stored service-account keys.

[G23] Terraform on Google Cloud overview

Declarative resource management, plans, providers, and state.

[G24] Execute Cloud Run jobs using Workflows

Official connector-based job orchestration.

[G25] Monitor Cloud Run health and performance

Built-in job metrics, log-based metrics, and alerting integration.

[D1] dbt BigQuery connection setup

Adapter profile configuration and supported authentication methods.

27 Repository and Validation Traceability

Primary source: the complete public repository at the pinned revision below. Every file was retrieved recursively and verified against its Git blob hash. Repository statements in this document reflect that revision, not an assumed latest branch or earlier audit report.

Commit: 42b0a7fba760119cc3f014191859f507208043fa

Inspection scope: Python orchestration/administration/acceptance code; PostgreSQL initialization and migrations; all dbt models, snapshots, macros, YAML contracts, and tests; fixtures; scripts; runtime/dependency files; Compose and configuration; documentation/ADRs; repository hygiene; historical remediation reports. No repository content was modified, and no acceptance execution, deployment, commit, or pull request was performed for this architecture document.

[R1] Platform scope and runtime

README.md

docs/requirements.md

docs/runtime_and_dependencies.md

pyproject.toml

docker-compose.yml

[R2] RAW and temporal delivery contracts

docs/raw_data_contracts.md

agridax/raw_contract.py

agridax/time_contract.py

infra/postgres/init/001_create_raw_schema.sql

[R3] Managed execution and processing boundary

docs/adr/0001_processing_and_single_writer.md

agridax/pipeline.py

agridax/database.py

infra/postgres/migrations/001_processing_control.sql

[R4] Effective and observed history

docs/adr/0002_temporal_states_and_revisions.md

dbt/agridax/snapshots/snap_fields.sql

[R5] Existing BigQuery target decision

docs/adr/0005_bigquery_target.md

[R6] Recovery and backfills

docs/recovery.md

agridax/pipeline.py

[R7] Workload privileges and dbt permissions

agridax/admin.py

dbt/agridax/dbt_project.yml

dbt/agridax/profiles.yml

[R8] Domain grains and transformations

docs/adr/0004_crop_and_metric_scope.md

dbt/agridax/models

dbt/agridax/macros

dbt/agridax/tests

[R9] Validated publication and acceptance

docs/adr/0003_stable_publication.md

agridax/publication.py

agridax/acceptance.py

Supporting evidence

[E1] AUDITORIA_FINAL_DEFINITIVA_AgriDax.md, supplied with the architecture request. It declares READY FOR FIRST COMMIT AND PUBLIC GITHUB, closes B01, and separates remaining non-blocking findings. Its current-state observations were checked against the inspected repository; its historical conclusions were not used as substitutes for code inspection.

[E2] acceptance_results.json, external native execution evidence supplied with the validated baseline. It reports scope native PostgreSQL acceptance, 142 PASS, 0 FAIL, and 0 NOT RUN, including authenticated/restricted roles, refused second writer, writer/ingestion locking, and old/new reader release behavior. It is validation evidence, not repository code and not a missing repository artifact.

The native evidence supports the local functional baseline. It does not validate BigQuery behavior, cloud IAM, GCP deployment, volume assumptions, or the proposed retention and concurrency protocols. Those require Phase 8 cloud acceptance.